

SUPPORT DE COURS COMPLET

Cryptographie

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction et principes fondamentaux
2. Chapitre 2 — Cryptographie symétrique : chiffrement par bloc et par flux
3. Chapitre 3 — Fonctions de hachage et authentification de message (MAC/HMAC)
4. Chapitre 4 — Cryptographie asymétrique : RSA, Diffie-Hellman, ECC
5. Chapitre 5 — Signatures numériques et infrastructures à clés publiques (PKI)
6. Chapitre 6 — Protocoles appliqués : TLS et cryptographie post-quantique

Cryptographie

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Comprendre les principes fondamentaux de la cryptographie moderne (confusion, diffusion, principe de Kerckhoffs).
- Maîtriser les primitives symétriques (chiffrement par bloc/flux, modes opératoires) et asymétriques (RSA, Diffie-Hellman, ECC).
- Comprendre les fonctions de hachage, les MAC/HMAC et les signatures numériques.
- Comprendre le fonctionnement d'une infrastructure à clés publiques (PKI) et du protocole TLS.
- Utiliser des bibliothèques cryptographiques standards de façon sûre, sans réimplémenter de primitives sensibles.

Prérequis

Des notions d'arithmétique modulaire facilitent la compréhension du chapitre 4 ; ce module est complémentaire du cours *Cryptanalyse*, qui étudie les attaques contre les mécanismes présentés ici.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction et principes fondamentaux	2h
2	Cryptographie symétrique : chiffrement par bloc et par flux	3h
3	Fonctions de hachage et authentification de message (MAC/HMAC)	2h
4	Cryptographie asymétrique : RSA, Diffie-Hellman, ECC	3h
5	Signatures numériques et infrastructures à clés publiques (PKI)	3h
6	Protocoles appliqués : TLS et cryptographie post-quantique	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Chiffrement symétrique avec OpenSSL	2h

Séance	Sujet	Durée
2	Hachage, HMAC et intégrité de fichiers	2h
3	RSA : génération de clés, chiffrement et signature	2h
4	Analyse d'une négociation TLS et des certificats	2h

Évaluation

- TP notés : 40 %
- Contrôle continu (exercices d'arithmétique modulaire et de manipulation de primitives) : 20 %
- Examen final : 40 %

Bibliographie

- J. Katz, Y. Lindell, *Introduction to Modern Cryptography*.
- A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*.
- Documentation OpenSSL — openssl.org/docs.
- NIST — publications FIPS 197 (AES), FIPS 180-4 (SHA-2), FIPS 186-5 (signatures numériques).

Chapitre 1 — Introduction et principes fondamentaux

1. Les objectifs de la cryptographie

La cryptographie moderne vise à garantir, en présence d'un adversaire potentiel, quatre propriétés fondamentales :

- **Confidentialité** : seules les parties autorisées peuvent accéder au contenu d'une information.
- **Intégrité** : toute modification non autorisée d'une information est détectable.
- **Authenticité** : l'origine d'une information (identité de l'émetteur) est vérifiable.
- **Non-répudiation** : l'émetteur d'un message ne peut pas nier ultérieurement en être l'auteur (propriété apportée notamment par la signature numérique).

Ces quatre propriétés ne sont pas systématiquement toutes requises en même temps. Un simple chiffrement symétrique d'un fichier personnel n'a par exemple besoin que de confidentialité. À l'inverse, un contrat signé électroniquement a surtout besoin d'intégrité, d'authenticité et de non-répudiation, et pas forcément de confidentialité (le contrat peut être public). Une partie du travail d'un architecte de sécurité consiste précisément à identifier, pour chaque flux d'information, quelles propriétés sont réellement nécessaires, car chacune a un coût (calcul, complexité de mise en œuvre, gestion de clés).

1.1 Illustration par le contexte

Scénario	Confidentialité	Intégrité	Authenticité	Non-répudiation
Message chiffré entre deux amis	nécessaire	souhaitable	souhaitable	non nécessaire
Mise à jour logicielle signée	non nécessaire (le code peut être public)	nécessaire	nécessaire	souhaitable
Contrat électronique	dépend du contexte	nécessaire	nécessaire	nécessaire
Trafic bancaire en ligne (TLS)	nécessaire	nécessaire	nécessaire	rarement exigée pour le client

Ce tableau illustre pourquoi aucun mécanisme unique ne couvre tous les besoins : c'est précisément l'articulation des primitives présentées dans les chapitres suivants (chiffrement symétrique, hachage, MAC, chiffrement asymétrique, signature) qui permet de composer la combinaison de propriétés adaptée à chaque situation.

2. Le principe de Kerckhoffs

Formulé par Auguste Kerckhoffs en 1883 : **la sécurité d'un système cryptographique ne doit reposer que sur le secret de la clé, jamais sur le secret de l'algorithme**. Un algorithme dont la sécurité dépend de rester confidentiel (« security through obscurity ») est fragile : il suffit qu'il soit

divulgué ou rétro-ingénié une seule fois pour effondrer la sécurité de tous les systèmes qui l'utilisent. À l'inverse, un algorithme public, largement étudié par la communauté scientifique (comme AES, RSA), gagne en confiance précisément parce qu'il résiste à un examen ouvert et prolongé — lien direct avec le module *Cryptanalyse*.

2.1 Pourquoi la publication renforce la confiance

Un algorithme cryptographique standardisé (AES, SHA-2, RSA, ECC) est publié avec sa spécification complète, ses vecteurs de test, et souvent son code de référence. Cela permet :

- à des équipes académiques et industrielles indépendantes de chercher des faiblesses pendant des années avant tout déploiement massif ;
- à des concours publics (comme celui qui a désigné Rijndael comme AES en 2001) de sélectionner l'algorithme le plus robuste parmi plusieurs candidats soumis à un examen collectif ;
- à l'écosystème (bibliothèques, matériel, protocoles) de converger vers un nombre restreint de primitives bien comprises plutôt qu'une multitude d'implémentations maison, chacune potentiellement fragile.

Un exemple historique bien documenté illustre l'inverse du principe de Kerckhoffs : l'algorithme A5/1, utilisé pour chiffrer les communications GSM, est resté secret pendant plusieurs années avant d'être rétro-ingénié ; une fois sa structure connue, des faiblesses structurelles ont pu être analysées publiquement. De même, l'algorithme de chiffrement CSS des DVD, conçu de façon fermée, a été cassé peu après sa divulgation. Ces cas illustrent concrètement pourquoi la communauté cryptographique privilégie aujourd'hui systématiquement des algorithmes ouverts et standardisés plutôt que des conceptions propriétaires non publiées.

2.2 Ce que Kerckhoffs n'interdit pas

Le principe ne dit pas qu'il faut publier sa clé, son mot de passe ou les détails d'une configuration précise (topologie réseau, version logicielle exacte). Il porte spécifiquement sur l'algorithme. Garder une configuration discrète relève de la défense en profondeur (module *Fondamentaux*), mais ne doit jamais être le fondement principal de la sécurité d'un mécanisme cryptographique.

3. Confusion et diffusion (Claude Shannon, 1949)

Deux propriétés structurantes de tout chiffrement robuste, formalisées par Claude Shannon dans son article fondateur *Communication Theory of Secrecy Systems* :

- **Confusion** : rendre la relation entre la clé et le texte chiffré la plus complexe possible, pour qu'une analyse statistique du chiffré ne révèle rien sur la clé.
- **Diffusion** : répartir l'influence de chaque bit du texte clair (et de la clé) sur l'ensemble du texte chiffré, pour qu'une modification d'un seul bit du clair change en moyenne la moitié des bits du chiffré (effet avalanche).

Ces deux notions justifient directement la structure des chiffrements par blocs modernes (tours de substitution et de permutation dans AES, chapitre 2). Sans confusion, un attaquant pourrait exploiter

une corrélation statistique simple entre clé et chiffré (comme dans un chiffrement par substitution monoalphabétique, cassable par analyse de fréquence). Sans diffusion, une petite modification du message clair ne modifierait qu'une petite portion prévisible du chiffré, facilitant des attaques différentielles.

3.1 Effet avalanche : illustration empirique

L'effet avalanche est plus facile à observer sur une fonction de hachage (chapitre 3) qu'à la main sur un chiffrement par bloc complet, mais le principe est identique. À titre d'illustration immédiate :

```
# Deux entrées ne différant que d'un caractère
echo -n "cours de securite" | sha256sum
echo -n "cours de securite." | sha256sum
```

Les deux empreintes produites n'ont statistiquement aucun bit en commun de façon prévisible, bien que les entrées ne diffèrent que d'un seul caractère : c'est la manifestation concrète de la diffusion. Un attaquant observant uniquement le chiffré (ou l'empreinte) ne peut donc rien inférer sur la nature de la modification apportée au clair.

4. Cryptographie symétrique vs asymétrique : vue d'ensemble

	Symétrique	Asymétrique
Clés	une seule clé, partagée entre émetteur et récepteur	une paire de clés (publique/privée)
Vitesse	rapide	beaucoup plus lente
Problème principal	distribution sécurisée de la clé partagée	vérification de l'authenticité de la clé publique
Exemples	AES, ChaCha20	RSA, Diffie-Hellman, ECC

En pratique, les systèmes réels combinent les deux : la cryptographie asymétrique établit ou échange une clé de session, ensuite utilisée pour un chiffrement symétrique rapide du volume de données (principe du **chiffrement hybride**, à l'œuvre notamment dans TLS, détaillé au chapitre 6).

4.1 Le problème combinatoire de la distribution de clés symétriques

La cryptographie asymétrique n'a pas été inventée par confort théorique : elle répond à un problème opérationnel concret. Si n personnes veulent pouvoir communiquer deux à deux de façon confidentielle en utilisant uniquement de la cryptographie symétrique, il faut une clé distincte pour chaque paire, soit :

```
nombre de clés =  $n(n-1)/2$ 
```

Pour illustrer avec des ordres de grandeur simples (exemple pédagogique) :

Nombre de participants (n)	Nombre de clés symétriques nécessaires
10	45
100	4 950
1 000	499 500

Ce nombre croît de façon quadratique : au-delà de quelques dizaines de participants, la distribution et le renouvellement manuel de toutes ces clés deviennent ingérables. La cryptographie asymétrique résout ce problème : chaque participant ne détient qu'**une seule paire de clés**, et diffuse librement sa clé publique — le nombre d'objets cryptographiques à gérer croît alors linéairement (n), et non plus quadratiquement.

5. Notion de sécurité prouvable et de sécurité calculatoire

La sécurité d'un système cryptographique moderne repose généralement sur une hypothèse calculatoire : un problème mathématique réputé difficile à résoudre en un temps raisonnable avec les moyens de calcul actuels et prévisibles (factorisation pour RSA, logarithme discret pour Diffie-Hellman/ECC). Il ne s'agit pas d'une preuve d'impossibilité absolue (à l'exception du **masque jetable** — *one-time pad* — seul système prouvé théoriquement incassable, mais impraticable car il exige une clé aussi longue que le message, utilisée une seule fois).

5.1 Niveaux de sécurité en bits

Pour comparer la robustesse de mécanismes de nature mathématique différente (chiffrement symétrique, factorisation, logarithme discret), on utilise la notion de **niveau de sécurité en bits** : un niveau de k bits signifie qu'un attaquant doit effectuer de l'ordre de 2^k opérations pour casser le mécanisme par la meilleure attaque connue. Le tableau suivant, cohérent avec les recommandations usuelles (NIST, ANSSI) évoquées au fil du module, donne une correspondance indicative entre familles d'algorithmes pour un même niveau de sécurité :

Niveau de sécurité	AES (symétrique)	RSA (modulus)	ECC (taille de courbe)
128 bits	AES-128	≈ 3072 bits	≈ 256 bits
192 bits	AES-192	≈ 7680 bits	≈ 384 bits
256 bits	AES-256	≈ 15360 bits	≈ 512 bits

Cette correspondance explique pourquoi les clés RSA sont numériquement bien plus grandes que les clés ECC ou AES à sécurité équivalente (rappel détaillé au chapitre 4) : la difficulté de factorisation d'un grand nombre croît beaucoup moins vite avec la taille de la clé que la difficulté d'une recherche exhaustive sur une clé symétrique.

5.2 Pourquoi 2^{128} est considéré comme hors de portée

Un ordre de grandeur pédagogique couramment cité pour illustrer l'infaisabilité d'une recherche exhaustive sur une clé de 128 bits : même en supposant un matériel capable de tester un très grand nombre de clés par seconde, parcourir l'espace des 2^{128} clés possibles prendrait un temps dépassant très largement toute échelle humaine ou même cosmologique envisageable avec la technologie actuelle. C'est cette marge de sécurité considérable, et non une preuve mathématique d'impossibilité, qui fonde la confiance accordée à AES-128 et aux mécanismes de niveau de sécurité comparable.

6. Modèles d'attaquant

L'analyse de sécurité d'un mécanisme cryptographique dépend explicitement de ce que l'on suppose que l'attaquant peut observer ou obtenir. Les modèles classiques, du plus faible au plus fort, sont :

- **Ciphertext-only** : l'attaquant n'a accès qu'à des textes chiffrés.
- **Known-plaintext** : l'attaquant connaît des couples (clair, chiffré) correspondants.
- **Chosen-plaintext (CPA)** : l'attaquant peut faire chiffrer des textes clairs de son choix et observer le résultat.
- **Chosen-ciphertext (CCA)** : l'attaquant peut en plus faire déchiffrer des textes chiffrés de son choix (sauf celui qu'il cherche à casser).

Un mécanisme moderne (AES-GCM, RSA-OAEP) est conçu pour résister au modèle le plus fort raisonnablement pertinent pour son usage, généralement CCA. Ce vocabulaire sera repris et approfondi au module *Cryptanalyse*.

7. Vocabulaire de base

Terme	Définition
Texte clair (plaintext)	message original, non protégé
Texte chiffré (ciphertext)	résultat du chiffrement
Clé	paramètre secret contrôlant le chiffrement/déchiffrement
Algorithme (chiffre, cipher)	procédure de transformation, publique (Kerckhoffs)
Cryptosystème	ensemble algorithme + gestion des clés + protocole d'usage
Nonce	valeur utilisée une seule fois avec une même clé, souvent non secrète
Vecteur d'initialisation (IV)	valeur d'amorçage d'un mode opératoire, souvent aléatoire
Sel (salt)	valeur aléatoire additionnelle, typiquement utilisée en hachage de mots de passe
Empreinte (digest)	sortie de taille fixe d'une fonction de hachage

Terme	Définition
Oracle	dans une analyse de sécurité, entité hypothétique répondant à des requêtes de l'attaquant (ex. « oracle de déchiffrement »)

8. Bonnes pratiques actuelles

- **Ne jamais concevoir son propre algorithme cryptographique** pour un usage réel : même des cryptographes expérimentés se trompent ; utiliser des standards éprouvés (AES, RSA/ECC, SHA-2/SHA-3) et des bibliothèques largement auditées (OpenSSL, libsodium, BoringSSL).
- **Générer tout matériel aléatoire (clés, nonces, sels) via un générateur cryptographiquement sûr (CSPRNG)**, jamais via une fonction pseudo-aléatoire générique (comme `rand()` en C) inadaptée à un usage cryptographique.
- **Suivre les paramètres recommandés par les référentiels actuels** (ANSSI, NIST) plutôt que des choix historiques ou par défaut potentiellement obsolètes.
- **Prévoir dès la conception l'agilité cryptographique** : la capacité à remplacer un algorithme ou une taille de clé sans réécrire toute l'architecture, en anticipation de futures dépréciations (illustré au chapitre 6 avec la transition post-quantique).

```
# Exemple : génération d'un matériel aléatoire de qualité cryptographique avec OpenSSL
openssl rand -hex 32          # 32 octets (256 bits) aléatoires, encodés en hexadécimal
openssl version -a           # vérifier la version d'OpenSSL et sa configuration
```

9. Pièges d'implémentation courants

- **« Rouler sa propre crypto »** : réinventer un algorithme ou un protocole plutôt que d'utiliser un composant standard et audité — source très fréquente de failles en pratique.
- **Comparaisons non constantes en temps** : comparer deux secrets (par exemple un MAC calculé et un MAC reçu) avec une comparaison naïve peut fuiter de l'information via le temps d'exécution (attaque temporelle, *timing attack*) ; il faut utiliser une fonction de comparaison en temps constant fournie par la bibliothèque cryptographique.
- **Confondre encodage et chiffrement** : Base64 n'apporte aucune confidentialité, ce n'est qu'un encodage réversible sans clé.
- **Clés codées en dur dans le code source** ou committées dans un dépôt de version, exposant durablement le secret même après correction.
- **Absence de plan de rotation des clés** : une clé compromise sans mécanisme de renouvellement prévu prolonge l'impact d'un incident bien au-delà du nécessaire.

À retenir

- Confidentialité, intégrité, authenticité, non-répudiation : quatre objectifs distincts, pas toujours tous nécessaires simultanément selon le contexte.
- Le principe de Kerckhoffs interdit de fonder la sécurité sur le secret de l'algorithme ; seule la clé doit rester secrète.

- Confusion et diffusion (Shannon) structurent la conception des chiffrements par blocs modernes et expliquent l'effet avalanche.
- La cryptographie asymétrique répond notamment au problème combinatoire de la distribution de clés symétriques à grande échelle.
- Le niveau de sécurité en bits permet de comparer des mécanismes de nature mathématique différente (symétrique, factorisation, logarithme discret).
- La cryptographie moderne combine en pratique symétrique (rapide, volume) et asymétrique (échange de clé, authentification) dans des schémas hybrides.
- Ne jamais concevoir sa propre cryptographie ; utiliser des standards publics, éprouvés, et des implémentations auditées.

Chapitre 2 — Cryptographie symétrique : chiffrement par bloc et par flux

1. Chiffrement par flux (stream cipher)

Combine le texte clair avec une suite pseudo-aléatoire (le « keystream ») générée à partir de la clé, généralement par XOR bit à bit :

```
chiffré[i] = clair[i] XOR keystream[i]
```

Exemple moderne largement utilisé : **ChaCha20**. Un chiffrement par flux exige impérativement de **ne jamais réutiliser le même couple (clé, nonce)** : réutiliser le même keystream sur deux messages différents permet de retrouver le XOR des deux messages clairs, une fuite d'information souvent exploitable (rappel du chapitre Cryptanalyse).

1.1 Pourquoi la réutilisation d'un keystream est catastrophique

Si un même keystream K est utilisé pour chiffrer deux messages clairs $m1$ et $m2$:

```
c1 = m1 XOR K
c2 = m2 XOR K
c1 XOR c2 = m1 XOR m2   (le keystream s'annule)
```

L'attaquant obtient directement le XOR des deux messages clairs sans jamais connaître la clé. Si l'un des deux messages est connu, ou si les deux présentent une structure exploitable (langage naturel, en-têtes de fichier prévisibles), il devient souvent possible de reconstituer une grande partie du contenu des deux messages. C'est exactement le mécanisme illustré, à titre d'exemple pédagogique, par le TP de cryptanalyse sur la réutilisation de flux chiffrant.

1.2 ChaCha20 : structure en bref

ChaCha20 génère son keystream à partir d'un état interne de 16 mots de 32 bits (clé de 256 bits, nonce, compteur de bloc), transformé par une série de tours mêlant additions, XOR et rotations (opérations dites « ARX » : Add-Rotate-XOR). Cette conception, purement logicielle et sans table de substitution, le rend particulièrement rapide et résistant aux attaques par canal auxiliaire basées sur le temps d'accès mémoire (contrairement à certaines implémentations non protégées d'AES en logiciel, avant la généralisation du support matériel AES-NI).

2. Chiffrement par bloc (block cipher)

Chiffre des blocs de taille fixe (128 bits pour AES) en appliquant plusieurs « tours » de transformations combinant substitution et permutation.

AES (Advanced Encryption Standard)

Standardisé par le NIST en 2001 (algorithme Rijndael), AES opère sur des blocs de 128 bits avec des clés de 128, 192 ou 256 bits, en 10, 12 ou 14 tours respectivement. Chaque tour comprend classiquement :

1. **SubBytes** : substitution non linéaire octet par octet (boîte-S), assure la confusion.
2. **ShiftRows** : décalage cyclique des lignes de l'état, contribue à la diffusion.
3. **MixColumns** : combinaison linéaire des colonnes (absente au dernier tour), renforce la diffusion.
4. **AddRoundKey** : XOR avec une sous-clé dérivée de la clé principale pour ce tour.

2.1 Cadencement de clé (key schedule)

AES ne réutilise pas directement la clé principale à chaque tour : un algorithme de **cadencement de clé** (*key schedule*) dérive, à partir de la clé initiale, une sous-clé distincte pour chaque tour (11 sous-clés pour AES-128, un tour supplémentaire de plus pour AES-192/256). Cette dérivation combine des rotations d'octets, des passages par la boîte-S, et des constantes de tour (*round constants*), afin que les sous-clés successives n'aient pas de relation linéaire simple entre elles — une relation trop simple faciliterait des attaques par cryptanalyse différentielle sur le cadencement lui-même.

2.2 AES-NI et implémentation matérielle

Les processeurs modernes (Intel, AMD, ARM) intègrent des instructions dédiées (AES-NI sur x86, instructions Crypto Extension sur ARM) qui exécutent directement les tours AES en matériel. Ceci apporte deux bénéfices simultanés : une exécution nettement plus rapide qu'une implémentation logicielle pure, et une résistance native aux attaques par canal auxiliaire temporel (l'accès aux boîtes-S ne dépend plus d'un tableau en mémoire dont le temps d'accès pourrait fuiter de l'information, comme cela pouvait être le cas pour certaines implémentations logicielles historiques d'AES).

```
# Vérifier la disponibilité d'AES-NI sur un système Linux
grep -o aes /proc/cpuinfo | head -n1

# Chiffrement/déchiffrement d'un fichier avec AES-256 en mode GCM via OpenSSL
openssl enc -aes-256-gcm -pbkdf2 -salt -in message.txt -out message.enc
openssl enc -aes-256-gcm -pbkdf2 -salt -d -in message.enc -out message_dechiffre.txt
```

3. Modes opératoires : pourquoi ils sont indispensables

Un chiffrement par bloc ne traite que des blocs de taille fixe : un **mode opératoire** définit comment enchaîner le chiffrement de plusieurs blocs pour traiter un message de longueur arbitraire.

Mode	Principe	Points clés
ECB (Electronic Codebook)	chaque bloc chiffré indépendamment	à proscrire : des blocs clairs identiques donnent des blocs chiffrés identiques (motifs visibles, cf. TP3 du module Cryptanalyse)

Mode	Principe	Points clés
CBC (Cipher Block Chaining)	chaque bloc clair est XORé avec le bloc chiffré précédent avant chiffrement ; nécessite un IV aléatoire	vulnérable aux attaques de padding oracle si le padding n'est pas vérifié de façon sûre
CTR (Counter)	transforme un chiffrement par bloc en chiffrement par flux, en chiffrant un compteur incrémental	parallélisable, mais un nonce réutilisé est aussi dangereux qu'en chiffrement par flux
GCM (Galois/Counter Mode)	mode CTR combiné à une authentification intégrée (AEAD)	recommandé par défaut : fournit confidentialité ET intégrité/authenticité en une seule primitive

3.1 Le cas ECB en détail

En mode ECB, chaque bloc de 128 bits identique du texte clair produit, avec la même clé, exactement le même bloc chiffré. Sur des données présentant une forte structure répétitive (une image bitmap non compressée, par exemple), les motifs du clair restent visibles dans le chiffré malgré le chiffrement — un exemple pédagogique classique montre une image chiffrée en ECB où le contour du logo d'origine reste parfaitement reconnaissable. C'est une démonstration directe et intuitive du fait qu'un mode opératoire mal choisi peut annuler une grande partie de la sécurité apportée par l'algorithme de bloc sous-jacent, aussi robuste soit-il.

3.2 CBC et le padding

Comme AES traite des blocs de 128 bits (16 octets), un message dont la longueur n'est pas un multiple exact de 16 octets doit être complété (« bourrage », *padding* — par exemple selon le schéma PKCS#7) avant chiffrement en CBC. Au déchiffrement, le padding est retiré et sa validité vérifiée. Si le système signale de façon distinguable (message d'erreur différent, ou même simplement un temps de réponse différent) qu'un padding est invalide, un attaquant peut exploiter cette information pour déchiffrer progressivement un message intercepté sans jamais connaître la clé — c'est l'attaque dite du **padding oracle**, une des raisons majeures pour lesquelles CBC est aujourd'hui déconseillé au profit des modes AEAD.

3.3 CTR : parallélisation

Le mode CTR chiffre indépendamment un compteur (dérivé du nonce et d'un index de bloc croissant) pour produire un keystream, XORé ensuite avec le clair — exactement comme un chiffrement par flux construit à partir d'un chiffrement par bloc. Chaque bloc étant indépendant des autres (contrairement à CBC), le chiffrement et le déchiffrement peuvent être parallélisés sur plusieurs cœurs de calcul, un avantage important pour le débit sur de gros volumes de données.

4. AEAD : chiffrement authentifié

Un mode **AEAD** (Authenticated Encryption with Associated Data), comme AES-GCM ou ChaCha20-Poly1305, combine chiffrement et code d'authentification en une seule opération, produisant un **tag d'authentification** vérifié au déchiffrement. Si le texte chiffré a été modifié (accidentellement ou par un attaquant), le déchiffrement échoue explicitement plutôt que de produire silencieusement un texte clair corrompu ou manipulé.

Recommandation pratique actuelle : privilégier systématiquement un mode AEAD (AES-GCM, ChaCha20-Poly1305) plutôt que de composer soi-même un mode CBC avec un HMAC séparé — une composition manuelle est une source fréquente d'erreurs d'implémentation.

4.1 Données associées authentifiées (AAD)

Un mode AEAD permet en plus d'authentifier des **données associées** (*Associated Data*) qui ne sont pas chiffrées mais dont l'intégrité doit être garantie conjointement au texte chiffré — par exemple un en-tête de paquet réseau en clair (adresse, numéro de séquence) qui doit rester lisible en transit mais ne doit pas pouvoir être altéré sans que la vérification échoue. C'est le rôle du paramètre **AAD** dans l'API GCM : il est intégré au calcul du tag d'authentification mais n'affecte pas le texte chiffré lui-même.

4.2 Structure interne de GCM (aperçu)

GCM combine un chiffrement en mode CTR (pour la confidentialité) avec une fonction d'authentification universelle appelée **GHASH**, construite sur une multiplication dans le corps fini $GF(2^{128})$. Le tag d'authentification résulte de ce calcul combinant texte chiffré, données associées et une sous-clé dérivée de la clé de chiffrement. Un point de vigilance technique important : la sécurité de GCM se dégrade fortement si le même couple (clé, nonce) est réutilisé, car cela permet potentiellement à un attaquant de retrouver la sous-clé d'authentification et de forger des tags valides pour d'autres messages — encore une illustration de la criticité de l'unicité du nonce (section 5).

```
# Illustration : chiffrement authentifié avec ChaCha20-Poly1305 (AEAD) via OpenSSL
openssl enc -chacha20-poly1305 -pbkdf2 -salt -in message.txt -out message.enc
```

5. Gestion de la clé et du vecteur d'initialisation (IV)/nonce

- La **clé** doit être générée par un générateur aléatoire cryptographiquement sûr (CSPRNG) et jamais codée en dur dans le code source.
- L'**IV/nonce** doit être unique pour chaque chiffrement avec une même clé (aléatoire pour CBC, souvent un compteur pour CTR/GCM) — sa réutilisation est l'une des erreurs d'implémentation les plus dommageables en pratique.

5.1 Nonce aléatoire vs nonce séquentiel

Deux stratégies courantes pour garantir l'unicité d'un nonce :

- **Nonce aléatoire** (généré par un CSPRNG à chaque chiffrement) : simple à mettre en œuvre côté client sans état persistant, mais expose à un risque de collision si l'espace de nonces est trop petit par rapport au nombre de messages chiffrés (paradoxe des anniversaires, rappel du

module *Cryptanalyse*) — pour GCM, la taille de nonce standard de 96 bits est conçue pour limiter ce risque tant qu'un volume raisonnable de messages est chiffré sous une même clé.

- **Nonce séquentiel** (compteur incrémental, souvent utilisé côté serveur avec état) : garantit l'unicité de façon déterministe tant que l'état du compteur est correctement persisté et jamais remis à zéro par erreur (par exemple après un redémarrage mal géré) — une remise à zéro accidentelle du compteur avec réutilisation de la même clé reproduit exactement le scénario dangereux décrit en section 1.1.

5.2 Rotation des clés

Aucune clé ne doit être utilisée indéfiniment : au-delà d'un certain volume de données chiffrées sous une même clé, la probabilité de collision de nonce ou d'exposition accrue à une éventuelle faiblesse cryptanalytique augmente. Les architectures robustes prévoient une politique de **rotation périodique des clés** (génération d'une nouvelle clé, réencodage ou transition progressive), combinée à un mécanisme d'identification de version de clé (souvent un identifiant de clé, ou *key ID*, associé aux données chiffrées) permettant de savoir avec quelle clé déchiffrer des données plus anciennes.

6. Dérivation de clé à partir d'un mot de passe

Un mot de passe humain n'a pas l'entropie suffisante pour servir directement de clé de chiffrement : on utilise une **fonction de dérivation de clé (KDF)** dédiée (PBKDF2, scrypt, Argon2), volontairement coûteuse en calcul, pour ralentir les attaques par force brute (rappel du chapitre *Cryptanalyse* sur le hachage de mots de passe).

6.1 Paramètres d'une KDF moderne

Une KDF moderne comme Argon2 expose plusieurs paramètres de coût, qu'il faut choisir en fonction du contexte (matériel disponible, contrainte de latence acceptable) :

- **Coût en temps** (nombre d'itérations internes) ;
- **Coût en mémoire** (quantité de mémoire vive requise pour le calcul, ce qui pénalise spécifiquement les attaques massivement parallélisées sur GPU/ASIC, moins efficaces sur des calculs gourmands en mémoire) ;
- **Parallélisme** (nombre de threads pouvant être utilisés).

```
# Dérivation d'une clé de chiffrement à partir d'un mot de passe (illustration OpenSSL,  
openssl enc -aes-256-gcm -pbkdf2 -iter 600000 -salt -in message.txt -out message.enc
```

Le **sel** (valeur aléatoire propre à chaque dérivation) est indispensable : sans lui, deux utilisateurs partageant le même mot de passe produiraient la même clé dérivée, et un attaquant pourrait précalculer une table de correspondances mot de passe → clé (attaque par table arc-en-ciel, détaillée au module *Cryptanalyse*).

7. Pièges d'implémentation courants

- **Réutiliser un IV/nonce** avec la même clé, en mode CBC comme en mode CTR/GCM — l'erreur la plus fréquente et la plus dommageable.
- **Choisir ECB par défaut** parce que c'est le mode le plus simple à utiliser dans certaines API historiques, sans en connaître les conséquences.
- **Ignorer l'échec de vérification du tag AEAD** au déchiffrement (par exemple continuer à traiter des données malgré une exception d'authentification non gérée) — cela annule totalement la protection d'intégrité apportée par l'AEAD.
- **Stocker la clé et les données chiffrées au même endroit** sans séparation des privilèges d'accès, réduisant le chiffrement à une protection purement cosmétique en cas de compromission du système de stockage.

À retenir

- AES est le standard de chiffrement par bloc de référence ; sa sécurité dépend fortement du mode opératoire choisi, pas seulement de l'algorithme lui-même.
- ECB est à proscrire ; les modes AEAD (GCM, ChaCha20-Poly1305) sont recommandés par défaut car ils apportent confidentialité et intégrité simultanément, avec authentification des données associées (AAD).
- La réutilisation d'un nonce/IV avec la même clé est l'une des erreurs d'implémentation les plus graves en cryptographie symétrique, qu'il s'agisse d'un chiffrement par flux ou d'un mode CTR/GCM.
- Une KDF (Argon2, scrypt, PBKDF2) est indispensable pour dériver une clé à partir d'un mot de passe ; le coût mémoire d'Argon2 le rend particulièrement résistant aux attaques matérielles massivement parallèles.
- La gestion opérationnelle des clés (génération sûre, rotation, séparation du stockage) est aussi déterminante pour la sécurité globale que le choix de l'algorithme.

Chapitre 3 — Fonctions de hachage et authentification de message (MAC/HMAC)

1. Fonctions de hachage cryptographiques

Une fonction de hachage H transforme une entrée de taille arbitraire en une sortie de taille fixe (l'empreinte ou *digest*), avec trois propriétés attendues (détaillées côté attaque au chapitre 4 du module *Cryptanalyse*) :

- résistance à la préimage,
- résistance à la seconde préimage,
- résistance aux collisions.

1.1 Reformulation intuitive des trois propriétés

- **Résistance à la préimage** : étant donné une empreinte h , il doit être impraticable de retrouver une entrée m telle que $H(m) = h$. C'est cette propriété qui permet, par exemple, de stocker une empreinte de mot de passe plutôt que le mot de passe lui-même.
- **Résistance à la seconde préimage** : étant donné une entrée m_1 , il doit être impraticable de trouver une entrée différente m_2 telle que $H(m_1) = H(m_2)$. Cette propriété protège contre la substitution d'un document par un autre ayant la même empreinte.
- **Résistance aux collisions** : il doit être impraticable de trouver une **paire quelconque** (m_1, m_2) telle que $H(m_1) = H(m_2)$, sans contrainte sur m_1 au départ. C'est la propriété la plus difficile à garantir, car le paradoxe des anniversaires (module *Cryptanalyse*) réduit la complexité théorique d'une recherche de collision à environ la racine carrée de l'espace de sortie, plutôt que sa totalité.

1.2 Panorama des fonctions usuelles

Fonction	Taille de sortie	Statut
MD5	128 bits	cassée (collisions pratiques), à ne plus utiliser pour un usage sécuritaire
SHA-1	160 bits	cassée (collision démontrée en 2017), dépréciée
SHA-2 (SHA-256, SHA-384, SHA-512)	256 à 512 bits	recommandée, aucune attaque pratique connue
SHA-3 (Keccak)	224 à 512 bits	recommandée, construction interne différente de SHA-2 (fonction éponge), diversifie le risque en cas de faille future sur la famille SHA-2

1.3 Construction de Merkle-Damgård (SHA-2) vs construction en éponge (SHA-3)

SHA-2 repose sur une **construction de Merkle-Damgård** : le message est découpé en blocs successifs, chacun étant combiné à l'état interne via une fonction de compression, l'état final constituant l'empreinte. Cette construction, bien comprise et éprouvée, présente néanmoins une propriété structurelle exploitable dans certains contextes : l'**attaque par extension de longueur** (*length extension attack*), qui permet, connaissant $H(m)$ sans connaître m , de calculer $H(m \parallel \text{padding} \parallel m')$ pour un suffixe m' choisi par l'attaquant, sans connaître m en entier. C'est précisément ce qui rend dangereuse une authentification naïve construite comme $H(\text{clé} \parallel \text{message})$ (voir section 4).

SHA-3 (Keccak) repose sur une **construction en éponge** (*sponge construction*), structurellement différente : les données sont absorbées puis « pressées » à travers une permutation interne, sans exposer directement un état intermédiaire réutilisable de la même manière. SHA-3 n'est donc pas vulnérable à l'attaque par extension de longueur, ce qui en fait une diversification utile, indépendamment de tout doute sur SHA-2 aujourd'hui.

```
# Calcul d'empreintes avec OpenSSL, comparaison de plusieurs fonctions
echo -n "cours de cryptographie" | openssl dgst -sha256
echo -n "cours de cryptographie" | openssl dgst -sha3-256
echo -n "cours de cryptographie" | openssl dgst -md5          # à des fins de comparaison u
```

2. Usages des fonctions de hachage

- **Vérification d'intégrité** : comparer l'empreinte d'un fichier téléchargé à une valeur de référence publiée.
- **Structures de données** : arbres de Merkle (utilisés en Blockchain, module suivant), tables de hachage.
- **Brique de construction** : dérivation de clé, génération de nombres pseudo-aléatoires, signatures numériques (chapitre 5), preuve de travail.
- **Hachage de mots de passe** : nécessite des fonctions dédiées et volontairement lentes (bcrypt, scrypt, Argon2), jamais une fonction générique seule (cf. module Cryptanalyse, chapitre 4).

2.1 Vérification d'intégrité en pratique

```
# Publication d'une empreinte de référence pour un fichier distribué
sha256sum archive.tar.gz > archive.tar.gz.sha256

# Vérification côté utilisateur après téléchargement
sha256sum -c archive.tar.gz.sha256
```

Ce mécanisme ne protège que contre une **altération accidentelle** (erreur de transmission) si l'empreinte de référence est publiée sur le même canal, non authentifié, que le fichier lui-même : un attaquant capable de modifier le fichier sur le serveur de distribution peut tout aussi bien modifier l'empreinte publiée en conséquence. Pour se protéger contre une altération **malveillante**, il faut authentifier l'empreinte elle-même — par exemple via une signature numérique (chapitre 5) ou en la publiant sur un canal distinct et de confiance.

2.2 Arbres de Merkle : principe

Un arbre de Merkle organise un grand ensemble de données en une structure arborescente où chaque nœud interne contient le hachage de la concaténation de ses deux enfants, jusqu'à une **racine de Merkle** unique qui résume l'ensemble de la structure. Toute modification, même d'une seule feuille, change la racine. Cette structure permet de vérifier efficacement qu'une donnée particulière appartient bien à un ensemble représenté par une racine connue, sans avoir à transmettre ni vérifier l'ensemble des données : il suffit d'un « chemin de preuve » de taille logarithmique par rapport au nombre total de feuilles. Ce principe est repris intensivement en Blockchain (module suivant) et dans les registres de transparence de certificats (Certificate Transparency, chapitre 5).

3. Code d'authentification de message (MAC)

Un MAC garantit à la fois l'**intégrité** d'un message et l'**authenticité** de son origine, pour deux parties partageant une clé secrète : $MAC = F_k(message)$, où seule une partie possédant la clé k peut produire ou vérifier un MAC valide.

À la différence d'une signature numérique (chapitre 5), un MAC ne fournit pas la non-répudiation : toute partie capable de vérifier le MAC est également capable d'en produire un (elle possède la même clé).

3.1 Vérification : comparaison en temps constant

Vérifier un MAC consiste à recalculer la valeur attendue et à la comparer à celle reçue. Une comparaison naïve, octet par octet, qui s'arrête dès le premier octet différent, laisse fuiter une information exploitable via le temps d'exécution : un attaquant peut, message après message, deviner le MAC correct un octet à la fois en mesurant de minuscules différences de temps de réponse (**attaque temporelle**, ou *timing attack*). Les bibliothèques cryptographiques fournissent pour cette raison des fonctions de comparaison en **temps constant** (par exemple `CRYPTO_memcmp` dans OpenSSL, ou `hmac.compare_digest` en Python), qui doivent systématiquement être utilisées à la place d'une comparaison directe (`==` ou `memcmp`) pour tout secret cryptographique.

4. HMAC (Hash-based MAC)

Construction générique permettant de bâtir un MAC à partir de n'importe quelle fonction de hachage :

$$HMAC(k, m) = H((k' \text{ XOR } opad) || H((k' \text{ XOR } ipad) || m))$$

où k' est la clé complétée à la taille de bloc de H , et `ipad/opad` des constantes fixes. Cette double application de H , avec la clé combinée de part et d'autre, neutralise spécifiquement l'attaque par extension de longueur qui rend dangereuse une simple concaténation $H(clé || message)$ (vue au chapitre 4 du module *Cryptanalyse*).

HMAC-SHA256 est aujourd'hui l'un des mécanismes de MAC les plus utilisés (authentification d'API, jetons JWT signés en HS256, intégrité de sessions).

4.1 Pourquoi $H(clé || message)$ seul est dangereux

Avec une construction Merkle-Damgård (SHA-2), connaître $H(\text{clé} || \text{message})$ permet à un attaquant, même sans connaître la clé ni le message, de calculer $H(\text{clé} || \text{message} || \text{padding} || \text{suffixe})$ pour un suffixe de son choix : il lui suffit de reprendre l'état interne révélé par l'empreinte connue et de continuer le calcul de hachage à partir de ce point, en profitant du padding déterministe de la construction. Concrètement, cela permettrait à un attaquant de forger un MAC valide pour un message différent du message original, en y ajoutant des données arbitraires à la fin, sans jamais connaître la clé secrète — un exemple documenté de ce type de faille a affecté des API web mal conçues utilisant un simple $H(\text{clé} || \text{message})$ comme mécanisme d'authentification de requêtes. HMAC évite ce piège précisément parce que la clé intervient à **deux endroits distincts et imbriqués** du calcul (via `ipad` et `opad`), rompant la propriété exploitée par l'attaque par extension de longueur.

```
# Calcul d'un HMAC-SHA256 avec OpenSSL
echo -n "message a authentifier" | openssl dgst -sha256 -hmac "cle_secrete_partagee"
```

4.2 HMAC et dérivation de clé (HKDF)

HMAC ne sert pas uniquement à authentifier des messages : il constitue également le cœur de **HKDF** (HMAC-based Key Derivation Function), une fonction de dérivation de clé standardisée utilisée notamment dans TLS 1.3 (chapitre 6) pour dériver, à partir d'un secret partagé issu d'un échange Diffie-Hellman, plusieurs clés symétriques indépendantes (clé de chiffrement, clé de MAC, etc.) de façon sûre et reproductible des deux côtés de la communication.

5. MAC intégré : GMAC et Poly1305

Les modes AEAD vus au chapitre précédent (AES-GCM, ChaCha20-Poly1305) intègrent directement un mécanisme de MAC (GMAC pour GCM, Poly1305 pour ChaCha20) dans l'opération de chiffrement elle-même, évitant d'avoir à composer manuellement chiffrement et authentification — une composition manuelle mal ordonnée (ex. authentifier avant de chiffrer plutôt que l'inverse) est une source d'erreurs classique.

5.1 Poly1305 : un MAC à usage unique par message

Poly1305 est un MAC basé sur l'évaluation d'un polynôme dans un corps fini de grande taille (nombre premier proche de 2^{130}), combiné à une clé à usage unique dérivée pour chaque message via ChaCha20. Sa conception, volontairement simple et rapide, offre une sécurité prouvable sous l'hypothèse que la clé à usage unique n'est effectivement jamais réutilisée pour deux messages différents — d'où l'importance, une fois de plus, de l'unicité stricte du nonce dans ChaCha20-Poly1305.

6. Ordre chiffrement/authentification (Encrypt-then-MAC)

Lorsqu'un schéma AEAD intégré n'est pas utilisé et qu'il faut composer soi-même chiffrement et MAC, la construction recommandée est **Encrypt-then-MAC** (chiffrer d'abord, puis authentifier le texte chiffré) plutôt que MAC-then-Encrypt ou Encrypt-and-MAC, car elle permet de rejeter un texte chiffré invalide

avant même de tenter de le déchiffrer, limitant la surface d'attaques par oracle (padding oracle notamment).

6.1 Comparaison des trois constructions

Construction	Définition	Risque principal
Encrypt-and-MAC	MAC calculé sur le clair, transmis à côté du chiffré	le MAC peut fuiter de l'information sur le clair, indépendamment du chiffrement
MAC-then-Encrypt	MAC calculé sur le clair, puis l'ensemble (message + MAC) est chiffré	le déchiffrement doit être tenté avant toute vérification, exposant à des attaques de type padding oracle si un mode par bloc avec padding est utilisé
Encrypt-then-MAC	le clair est chiffré, puis le MAC est calculé sur le texte chiffré	recommandé : le MAC est vérifié avant tout déchiffrement, un texte chiffré invalide est rejeté immédiatement

Les modes AEAD (section 5) implémentent en réalité une forme optimisée et intégrée d'Encrypt-then-MAC, ce qui explique pourquoi ils sont recommandés par défaut plutôt que de recomposer manuellement ces trois briques séparément.

7. Pièges d'implémentation courants

- **Utiliser une fonction de hachage générique (SHA-256) seule pour stocker des mots de passe**, sans sel ni ralentissement volontaire : une fonction de hachage rapide permet à un attaquant de tester des milliards de mots de passe candidats par seconde sur du matériel dédié — il faut une fonction dédiée (bcrypt, scrypt, Argon2).
- **Comparer des MAC ou des empreintes avec une comparaison non constante en temps**, exposant à une attaque temporelle (section 3.1).
- **Construire une authentification maison via `H(clé || message)`** au lieu de HMAC, vulnérable à l'attaque par extension de longueur sur les fonctions de type Merkle-Damgård.
- **Publier une empreinte d'intégrité sur le même canal non authentifié que le fichier associé**, n'apportant alors qu'une protection contre la corruption accidentelle, pas contre une altération malveillante.

À retenir

- Une fonction de hachage cryptographique doit résister à la préimage, à la seconde préimage et aux collisions ; SHA-2 et SHA-3 sont recommandées, MD5 et SHA-1 sont cassées.
- La construction en éponge de SHA-3 la rend, contrairement à SHA-2, naturellement résistante à l'attaque par extension de longueur.
- Un MAC garantit intégrité et authenticité pour des parties partageant une clé secrète, sans non-répudiation.
- HMAC neutralise spécifiquement l'attaque par extension de longueur inhérente aux fonctions de hachage basées sur Merkle-Damgård, en intégrant la clé à deux endroits du calcul.

- Privilégier un mode AEAD intégré (GCM, Poly1305) plutôt que de composer manuellement chiffrement et MAC ; si nécessaire, appliquer le principe Encrypt-then-MAC.
- Toute comparaison de secret cryptographique (MAC, empreinte, jeton) doit être effectuée en temps constant pour éviter les attaques temporelles.

Chapitre 4 — Cryptographie asymétrique : RSA, Diffie-Hellman, ECC

1. Principe général

Chaque partie dispose d'une paire de clés mathématiquement liées : une **clé publique**, diffusable librement, et une **clé privée**, gardée strictement secrète. Deux usages principaux : chiffrer avec la clé publique du destinataire (lui seul peut déchiffrer avec sa clé privée), ou signer avec sa propre clé privée (tout le monde peut vérifier avec la clé publique correspondante — chapitre 5).

Cette asymétrie fondamentale repose sur la notion de **fonction à sens unique avec trappe** (*trapdoor one-way function*) : une fonction facile à calculer dans un sens, difficile à inverser sans une information secrète particulière (la trappe), mais facile à inverser avec cette information. Multiplier deux grands nombres premiers est facile ; retrouver les deux facteurs à partir du produit est difficile — sauf à connaître déjà l'un des deux facteurs (RSA). Calculer $g^a \bmod p$ est facile ; retrouver a à partir de $g^a \bmod p$ est difficile (Diffie-Hellman, ECC).

2. Rappels d'arithmétique modulaire nécessaires

- **Congruence modulo n** : $a \equiv b \pmod{n}$ si a et b ont le même reste dans la division par n .
- **Exponentiation modulaire** : $a^e \bmod n$, calculable efficacement par l'algorithme d'exponentiation rapide (« square and multiply »), sans jamais calculer a^e en entier.
- **Petit théorème de Fermat / théorème d'Euler** : fondement de la construction de RSA.
- **Inverse modulaire** : d tel que $e \times d \equiv 1 \pmod{\varphi(n)}$, calculable par l'algorithme d'Euclide étendu — c'est ainsi que la clé privée RSA est dérivée de la clé publique et des facteurs premiers.

2.1 Exponentiation rapide : illustration simplifiée

Calculer $a^{13} \bmod n$ naïvement demanderait 12 multiplications successives. L'algorithme « square and multiply » exploite l'écriture binaire de l'exposant ($13 = 1101$ en binaire) pour ne réaliser qu'un nombre de multiplications proportionnel au nombre de bits de l'exposant :

```
13 en binaire : 1101
a^13 = a^8 × a^4 × a^1
      = ((a^2)^2)^2 × (a^2)^2 × a
```

Ce principe, appliqué à des exposants de plusieurs centaines de chiffres, est ce qui rend RSA praticable en temps raisonnable malgré la taille considérable des nombres manipulés.

2.2 Un exemple arithmétique jouet (à but pédagogique uniquement, tailles non sécurisées)

À titre purement illustratif — ces tailles de nombres sont bien trop petites pour un usage réel, elles serviraient à un adversaire disposant d'un simple crayon :

```
p = 61, q = 53
n = p × q = 3233
φ(n) = (p-1)(q-1) = 60 × 52 = 3120
e = 17 (choisi premier avec 3120)
d = 2753 (inverse modulaire de 17 modulo 3120)

Chiffrement d'un message m = 65 :
c = 6517 mod 3233 = 2790

Déchiffrement :
m = 27902753 mod 3233 = 65
```

Cet exemple (classique dans la littérature pédagogique sur RSA) illustre la mécanique exacte de l'algorithme sur des nombres suffisamment petits pour être vérifiables, mais il ne doit jamais laisser penser que des tailles de cet ordre seraient utilisables en pratique : la sécurité de RSA repose sur l'usage de nombres premiers p et q de plusieurs centaines de chiffres.

3. RSA : construction

1. Choisir deux grands nombres premiers distincts p et q ; calculer $n = p \times q$.
2. Calculer $\varphi(n) = (p-1)(q-1)$ (indicatrice d'Euler).
3. Choisir un exposant public e premier avec $\varphi(n)$ (souvent 65537).
4. Calculer l'exposant privé d , inverse modulaire de e modulo $\varphi(n)$.
5. Clé publique : (n, e) . Clé privée : (n, d) .

Chiffrement : $c = m^e \bmod n$. **Déchiffrement** : $m = c^d \bmod n$.

La sécurité repose sur la difficulté de retrouver d (ou de factoriser n) sans connaître p et q (rappel : chapitre 5 du module *Cryptanalyse*).

3.1 Pourquoi 65537 ?

L'exposant public 65537 (soit $2^{16} + 1$) est un choix très répandu car il combine deux avantages : son écriture binaire (1000000000000001) ne comporte que deux bits à 1, ce qui rend l'exponentiation modulaire $m^e \bmod n$ particulièrement rapide au chiffrement ou à la vérification de signature (peu de multiplications nécessaires), tout en étant suffisamment grand pour éviter certaines attaques structurelles connues sur RSA lorsque l'exposant public est trop petit (comme $e = 3$ combiné à un padding insuffisant).

Padding : une nécessité, pas une option

RSA « brut » (sans padding) est déterministe et vulnérable à plusieurs attaques structurelles (cf. module *Cryptanalyse*). Les standards actuels imposent un schéma de padding : **OAEP** pour le chiffrement, **PSS** pour la signature (chapitre 5). Utiliser RSA sans padding correct est une erreur d'implémentation grave, quelle que soit la taille de clé.

3.2 OAEP en bref

OAEP (Optimal Asymmetric Encryption Padding) introduit de l'aléa dans le message avant chiffrement RSA, de sorte que chiffrer deux fois le même message clair produise deux textes chiffrés différents (propriété dite de **sécurité sémantique**, absente de RSA brut qui est déterministe). OAEP combine le message avec une valeur aléatoire à travers une construction en deux tours utilisant des fonctions de hachage, rendant également le schéma résistant à des attaques par texte chiffré choisi lorsqu'il est correctement implémenté.

```
# Génération d'une paire de clés RSA (2048 bits) et chiffrement avec padding OAEP via openssl
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out rsa_prive.pem
openssl pkey -in rsa_prive.pem -pubout -out rsa_publique.pem

openssl pkeyutl -encrypt -pubin -inkey rsa_publique.pem \
  -pkeyopt rsa_padding_mode:oaep -in message.txt -out message.enc

openssl pkeyutl -decrypt -inkey rsa_prive.pem \
  -pkeyopt rsa_padding_mode:oaep -in message.enc -out message_dechiffre.txt
```

4. Échange de clé Diffie-Hellman

Permet à deux parties d'établir un secret partagé sur un canal public, sans jamais transmettre ce secret directement :

1. Paramètres publics : un nombre premier p et un générateur g .
2. Alice choisit un secret a , calcule et envoie $A = g^a \bmod p$.
3. Bob choisit un secret b , calcule et envoie $B = g^b \bmod p$.
4. Alice calcule $B^a \bmod p$, Bob calcule $A^b \bmod p$: les deux obtiennent la même valeur $g^{(ab)} \bmod p$, le secret partagé.

Un observateur du canal voit p , g , A et B , mais retrouver a ou b (ou directement $g^{(ab)}$) requiert de résoudre le problème du logarithme discret, réputé difficile pour des paramètres suffisamment grands.

Limite importante : Diffie-Hellman « pur » ne fournit aucune authentification — il est vulnérable à une attaque de l'homme du milieu (*man-in-the-middle*) si les parties ne vérifient pas par ailleurs l'identité de leur interlocuteur (rôle des certificats et signatures, chapitre 5).

4.1 Exemple arithmétique jouet

```
Paramètres publics : p = 23, g = 5

Alice choisit a = 6 → A = 5^6 mod 23 = 8      (envoyé à Bob)
Bob choisit b = 15 → B = 5^15 mod 23 = 19     (envoyé à Alice)

Alice calcule : B^a mod p = 19^6 mod 23 = 2
Bob calcule   : A^b mod p = 8^15 mod 23 = 2

Secret partagé obtenu par les deux parties : 2
```

Comme pour RSA, ces tailles sont purement pédagogiques : les déploiements réels utilisent des groupes de plusieurs centaines à plusieurs milliers de bits pour le Diffie-Hellman classique, ou des courbes elliptiques normalisées pour ECDH (section 5).

4.2 Diffie-Hellman éphémère (DHE / ECDHE)

Dans une variante dite **éphémère**, chaque partie génère une nouvelle paire (a, A) ou (b, B) pour chaque session, puis la détruit une fois le secret partagé calculé, au lieu de réutiliser des paramètres fixes sur le long terme. C'est ce mécanisme qui fournit la propriété de **confidentialité persistante** (*Perfect Forward Secrecy*), détaillée au chapitre 6 : même si la clé privée long terme d'un serveur est compromise ultérieurement, les secrets de session éphémères passés, déjà détruits, ne peuvent pas être reconstitués rétroactivement.

5. Cryptographie sur courbes elliptiques (ECC)

Remplace le groupe multiplicatif modulo p par le groupe des points d'une courbe elliptique définie sur un corps fini. Le problème équivalent au logarithme discret (ECDLP) y est considéré plus difficile à taille de clé égale, permettant des clés bien plus courtes pour un niveau de sécurité équivalent (rappel chapitre 5 du module *Cryptanalyse* : ECC 256 bits $\approx$ RSA 3072 bits). Courbes largement utilisées aujourd'hui : P-256 (NIST), Curve25519 (utilisée par X25519 pour l'échange de clé et Ed25519 pour la signature, chapitre 5).

5.1 Intuition géométrique

Une courbe elliptique est définie par une équation de la forme $y^2 = x^3 + ax + b$, et l'on définit une opération d'« addition » entre deux points de la courbe, géométriquement construite en traçant une droite passant par les deux points et en réfléchissant son troisième point d'intersection avec la courbe par rapport à l'axe des abscisses. Cette opération d'addition dote l'ensemble des points de la courbe (plus un point « à l'infini » servant d'élément neutre) d'une structure de groupe mathématique. La « multiplication scalaire » $k \times P$ (ajouter un point P à lui-même k fois, calculée efficacement par une méthode analogue au « square and multiply ») joue, dans ce groupe, le rôle que joue l'exponentiation modulaire dans Diffie-Hellman classique — d'où le nom d'**ECDH** (Elliptic Curve Diffie-Hellman) pour l'échange de clé sur courbe elliptique, qui suit exactement le même schéma que la section 4, en remplaçant $g^a \bmod p$ par $a \times G$ (G étant un point générateur public fixé par la courbe).

5.2 Pourquoi des clés plus courtes suffisent

La meilleure attaque connue contre le logarithme discret classique (modulo p) exploite des algorithmes sous-exponentiels (comme le crible algébrique généralisé, utilisé aussi pour factoriser un module RSA), ce qui oblige à utiliser des paramètres numériquement très grands pour atteindre un niveau de sécurité donné. Sur une courbe elliptique bien choisie, en revanche, la meilleure attaque générique connue (méthode « rho » de Pollard adaptée) reste pleinement exponentielle en fonction de la taille de la clé, sans raccourci sous-exponentiel connu — d'où des tailles de clé nettement plus courtes pour un niveau de sécurité équivalent, comme rappelé dans le tableau du chapitre 1.

```
# Génération d'une paire de clés ECC (courbe P-256) avec OpenSSL
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out ec_prive.pem
openssl pkey -in ec_prive.pem -pubout -out ec_publique.pem

# Génération d'une paire de clés X25519 (échange de clé moderne)
openssl genpkey -algorithm X25519 -out x25519_prive.pem
```

5.3 Un piège classique : le choix de la courbe

Toutes les courbes elliptiques ne se valent pas. Certaines courbes historiques présentent des propriétés qui facilitent des attaques (courbes dites « anormales », ou choix de paramètres mal justifiés publiquement). C'est pourquoi il est recommandé de n'utiliser que des courbes standardisées et largement examinées par la communauté (P-256/P-384 du NIST, Curve25519/Curve448), plutôt que des paramètres de courbe choisis sans justification publique claire — un rappel direct du principe de Kerckhoffs (chapitre 1) appliqué au choix des paramètres, pas seulement de l'algorithme.

6. Chiffrement hybride

En pratique, la cryptographie asymétrique n'est presque jamais utilisée pour chiffrer directement de gros volumes de données (trop lente) : elle sert à établir ou transmettre une **clé de session**, ensuite utilisée avec un chiffrement symétrique rapide (AES-GCM). C'est le principe exact mis en œuvre par TLS (chapitre 6).

6.1 Deux façons d'obtenir la clé de session

- **Transport de clé** (schéma historique, ex. RSA) : l'émetteur génère lui-même une clé symétrique aléatoire, puis la chiffre avec la clé publique RSA du destinataire. Inconvénient majeur : la compromission ultérieure de la clé privée RSA permet de déchiffrer rétroactivement toutes les sessions passées enregistrées, car la même clé privée sert à toutes les sessions.
- **Accord de clé** (schéma recommandé aujourd'hui, ex. ECDHE) : les deux parties calculent conjointement un secret partagé via un échange Diffie-Hellman éphémère, sans jamais transmettre la clé de session elle-même sur le canal. Combiné à l'usage de paramètres éphémères (section 4.2), ce schéma apporte la confidentialité persistante, absente du transport de clé RSA classique — l'une des raisons pour lesquelles TLS 1.3 a supprimé le transport de clé RSA (chapitre 6).

À retenir

- RSA repose sur la difficulté de factorisation ; Diffie-Hellman et ECC reposent sur la difficulté du logarithme discret (classique ou sur courbe elliptique).
- Le padding (OAEP/PSS) n'est pas optionnel : RSA brut est déterministe et vulnérable à des attaques structurelles connues ; OAEP apporte l'aléa nécessaire à la sécurité sémantique.
- Diffie-Hellman pur n'authentifie pas les parties ; il doit être combiné à un mécanisme d'authentification pour résister à une attaque de l'homme du milieu.

- La variante éphémère de Diffie-Hellman (DHE/ECDHE) apporte la confidentialité persistante, absente du transport de clé RSA classique.
- ECC atteint un niveau de sécurité équivalent à RSA avec des clés beaucoup plus courtes, grâce à l'absence de raccourci sous-exponentiel connu contre le logarithme discret sur courbe elliptique bien choisie.
- Le choix des paramètres (courbe elliptique, groupe Diffie-Hellman) doit toujours s'appuyer sur des standards publiquement examinés, jamais sur des paramètres non justifiés.

Chapitre 5 — Signatures numériques et infrastructures à clés publiques (PKI)

1. Principe de la signature numérique

La signature numérique fournit **authenticité**, **intégrité** et **non-répudiation** : le signataire utilise sa clé privée pour produire une signature liée à un message, que quiconque peut vérifier avec la clé publique correspondante, sans pouvoir produire lui-même une signature valide.

Schéma général

1. Calculer l'empreinte du message : $h = H(\text{message})$ (fonction de hachage, chapitre 3).
2. Signer l'empreinte avec la clé privée : $\text{signature} = \text{Sign}(\text{clé privée}, h)$.
3. Vérification : recalculer $H(\text{message})$ et vérifier la signature avec la clé publique.

Signer l'empreinte plutôt que le message entier permet de traiter des messages de taille arbitraire avec des opérations cryptographiques de taille fixe, et rend toute modification du message détectable (l'empreinte changerait).

1.1 Non-répudiation : ce qu'elle apporte concrètement par rapport à un MAC

Rappel du chapitre 3 : un MAC prouve qu'un message provient d'une partie possédant la clé secrète partagée, mais comme les deux parties (émetteur et vérificateur) possèdent cette même clé, aucune des deux ne peut prouver, face à un tiers, laquelle des deux a réellement produit le MAC. Avec une signature numérique, seule la clé **privée**, détenue par une seule partie, permet de produire une signature valide ; la clé **publique**, utilisée pour la vérification, ne permet à personne de forger une signature. Un tiers (par exemple un juge, ou un système d'audit) peut donc vérifier une signature avec la seule clé publique et en déduire, avec une confiance justifiée, que seule la partie détenant la clé privée correspondante a pu la produire — c'est cette asymétrie qui fonde la non-répudiation.

1.2 Signature avec récupération de message vs signature avec appendice

La plupart des schémas modernes (RSA-PSS, ECDSA, Ed25519) sont des **signatures avec appendice** : la signature est une valeur distincte, transmise à côté du message, et sa vérification requiert de disposer du message complet pour en recalculer l'empreinte. Il existe aussi historiquement des schémas à **récupération de message**, où le message (ou une partie) peut être reconstitué directement à partir de la signature elle-même ; ces schémas restent marginaux en pratique face aux schémas à appendice, plus flexibles pour des messages de taille arbitraire.

2. RSA-PSS

Schéma de signature basé sur RSA avec un padding probabiliste (PSS — Probabilistic Signature Scheme), aujourd'hui recommandé pour la signature (par opposition à un ancien schéma déterministe

PKCS#1 v1.5, encore présent mais moins recommandé pour de nouveaux déploiements).

2.1 Pourquoi « probabiliste » est préférable à « déterministe » pour signer

PSS introduit un sel aléatoire dans le calcul de la signature, de sorte que signer deux fois le même message avec la même clé privée produit deux signatures différentes (mais toutes deux valides à la vérification). Ce comportement probabiliste renforce la sécurité de la preuve mathématique associée au schéma (résistance démontrée sous des hypothèses standards, dans un cadre de preuve de sécurité appelé « modèle de l'oracle aléatoire »), alors que PKCS#1 v1.5, plus ancien et déterministe, s'appuie sur une analyse de sécurité moins directe, bien qu'aucune attaque pratique rédhibitoire ne soit connue contre une implémentation correcte pour la signature (la situation est différente pour le chiffrement RSA avec PKCS#1 v1.5, où des attaques structurelles bien documentées existent, d'où l'exigence d'OAEP au chapitre 4).

```
# Génération d'une paire de clés RSA, signature et vérification avec RSA-PSS via OpenSSL
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out rsa_prive.pem
openssl pkey -in rsa_prive.pem -pubout -out rsa_publique.pem

openssl dgst -sha256 -sign rsa_prive.pem -sigopt rsa_padding_mode:pss \
  -out message.sig message.txt

openssl dgst -sha256 -verify rsa_publique.pem -sigopt rsa_padding_mode:pss \
  -signature message.sig message.txt
```

3. ECDSA et Ed25519

- **ECDSA** : signature sur courbe elliptique, largement déployée (certificats TLS, Bitcoin). Point de vigilance : la sécurité d'ECDSA dépend crucialement d'un nombre aléatoire (« nonce ») unique et non prévisible à chaque signature — un nonce réutilisé ou prévisible permet de retrouver directement la clé privée (illustré par des compromissions réelles, notamment sur certaines consoles de jeu et portefeuilles Bitcoin mal implémentés).
- **Ed25519** : schéma de signature moderne basé sur la courbe Curve25519, déterministe (ne dépend pas d'un aléa externe à chaque signature, éliminant le risque de nonce faible), rapide et largement recommandé pour les nouveaux déploiements (SSH, protocoles modernes).

3.1 Pourquoi un nonce ECDSA faible expose directement la clé privée

Sans entrer dans le détail complet de la construction, l'intuition est la suivante : la signature ECDSA combine la clé privée d et un nonce secret k de telle façon que si deux signatures différentes sont produites avec le **même** k (ou avec des valeurs de k liées par une relation connue de l'attaquant, par exemple issues d'un générateur pseudo-aléatoire de mauvaise qualité), un simple système d'équations permet de retrouver k , puis d , à partir des deux signatures et des messages correspondants — sans jamais casser directement le logarithme discret sous-jacent. C'est exactement le mécanisme exploité dans des compromissions réelles et documentées : une console de jeu grand public dont l'implémentation ECDSA réutilisait un nonce fixe au lieu d'un nonce aléatoire a ainsi vu sa clé de signature de firmware extraite ; plusieurs portefeuilles de cryptomonnaie utilisant un générateur

aléatoire défaillant sur certains environnements mobiles ont subi le même type de compromission, avec un impact financier direct pour les utilisateurs concernés.

3.2 Signature déterministe : deux approches pour éviter ce piège

- **RFC 6979** : dérive un nonce ECDSA de façon déterministe à partir de la clé privée et du message (via HMAC), en éliminant la dépendance à un générateur aléatoire externe défaillant au moment de la signature, tout en restant compatible avec ECDSA standard.
- **Ed25519** : élimine le problème par construction même du schéma, le nonce étant systématiquement dérivé de façon déterministe et interne à l'algorithme dès sa conception, sans configuration supplémentaire requise de la part de l'implémenteur — un facteur important dans sa popularité pour les nouveaux déploiements (clés SSH, signatures de paquets logiciels, protocoles comme WireGuard).

```
# Génération d'une paire de clés Ed25519, signature et vérification avec OpenSSL
openssl genpkey -algorithm ED25519 -out ed25519_prive.pem
openssl pkey -in ed25519_prive.pem -pubout -out ed25519_publique.pem

openssl pkeyutl -sign -inkey ed25519_prive.pem -rawin -in message.txt -out message.sig
openssl pkeyutl -verify -pubin -inkey ed25519_publique.pem -rawin -in message.txt -sigf
```

4. Certificats numériques

Un certificat numérique lie une identité (personne, serveur, organisation) à une clé publique, la liaison étant elle-même garantie par la signature numérique d'une **autorité de certification (AC/CA)** de confiance. Format standard le plus répandu : **X.509**.

Contenu typique d'un certificat X.509

- Identité du sujet (ex. nom de domaine pour un certificat de serveur).
- Clé publique du sujet.
- Identité de l'autorité de certification émettrice.
- Période de validité.
- Signature numérique de l'AC sur l'ensemble de ces informations.

4.1 Extensions courantes d'un certificat X.509

Au-delà des champs de base, un certificat X.509 moderne comporte des **extensions** qui précisent son usage :

- **Subject Alternative Name (SAN)** : liste des noms de domaine (ou adresses IP) couverts par le certificat ; les navigateurs modernes vérifient ce champ en priorité pour valider un nom de domaine, plutôt que l'ancien champ *Common Name*.
- **Key Usage / Extended Key Usage** : précise les usages autorisés de la clé (signature de certificat, authentification serveur, authentification client, signature de code, etc.), limitant le risque qu'un certificat émis pour un usage soit détourné vers un autre.

- **Basic Constraints** : indique si le certificat peut lui-même signer d'autres certificats (certificat d'AC) ou non (certificat final, dit « feuille »), une distinction essentielle pour la chaîne de confiance (section 5).

```
# Inspection du contenu d'un certificat X.509 avec OpenSSL
openssl x509 -in certificat.pem -noout -text

# Vérification d'un certificat serveur en se connectant directement en TLS
openssl s_client -connect exemple.org:443 -servername exemple.org </dev/null
```

5. Infrastructure à clés publiques (PKI)

Ensemble des composants et procédures permettant de créer, distribuer, vérifier et révoquer des certificats :

- **Autorité de certification (AC/CA)** : émet et signe les certificats.
- **Autorité d'enregistrement (RA)** : vérifie l'identité du demandeur avant émission.
- **Chaîne de confiance** : un certificat intermédiaire est lui-même signé par une AC racine, dont la clé publique est préinstallée (« ancre de confiance ») dans les navigateurs et systèmes d'exploitation.
- **Révocation** : mécanismes de liste de révocation (CRL) ou de vérification en ligne (OCSP), permettant d'invalidier un certificat compromis avant sa date d'expiration normale.

5.1 Pourquoi une chaîne intermédiaire plutôt qu'une signature directe par l'AC racine

Les AC racines ne signent en pratique presque jamais directement les certificats des serveurs finaux. Leur clé privée, extrêmement sensible, est conservée hors ligne dans des conditions de sécurité physique et procédurale très strictes, et n'est utilisée que pour signer un nombre restreint de **certificats d'AC intermédiaires**. Ce sont ces AC intermédiaires, dont la clé privée reste en ligne pour un usage opérationnel, qui signent au quotidien les certificats des serveurs finaux. Ce découplage limite fortement les conséquences d'une éventuelle compromission d'une AC intermédiaire (elle peut être révoquée sans remettre en cause la racine elle-même) et illustre un principe de défense en profondeur appliqué directement à la gestion de clés (module *Fondamentaux*).

5.2 CRL vs OCSP

- **CRL** (Certificate Revocation List) : liste, publiée périodiquement par l'AC, de tous les numéros de série des certificats révoqués avant leur expiration normale. Le client doit télécharger cette liste (potentiellement volumineuse) et vérifier localement la présence du certificat concerné.
- **OCSP** (Online Certificate Status Protocol) : le client interroge directement un serveur de l'AC pour connaître le statut d'un certificat précis, en temps quasi réel, sans avoir à télécharger une liste complète. Une variante, **OCSP stapling**, permet au serveur lui-même de joindre une réponse OCSP récente et signée directement lors de la poignée de main TLS, évitant au client une requête réseau supplémentaire vers l'AC et réduisant l'exposition d'information sur les sites visités par le client.

Vérification d'un certificat côté client

1. Le certificat est-il signé par une AC de confiance (remontée de la chaîne jusqu'à une ancre de confiance) ?
2. Le certificat est-il dans sa période de validité ?
3. Le certificat correspond-il bien à l'identité attendue (ex. nom de domaine) ?
4. Le certificat a-t-il été révoqué ?

Une faille dans n'importe laquelle de ces vérifications (ex. accepter un certificat expiré, ou ne pas vérifier le nom de domaine) annule la protection apportée par toute la chaîne cryptographique — un point classique d'audit de sécurité applicative (module *Audit organisation et technique*, chapitre 5).

6. Modèles alternatifs

- **Web of Trust** (utilisé historiquement par PGP/GPG) : confiance distribuée entre pairs plutôt que hiérarchique, sans autorité centrale.
- **Certificate Transparency** : registres publics et vérifiables des certificats émis, permettant de détecter des émissions frauduleuses par une AC compromise ou négligente.

6.1 Web of Trust : principe et limites

Dans le modèle du Web of Trust, chaque utilisateur signe lui-même les clés publiques d'autres utilisateurs dont il a personnellement vérifié l'identité (par exemple lors d'une rencontre en personne, une pratique parfois formalisée sous le nom de « key signing party »). La confiance accordée à une clé inconnue dépend alors du réseau de signatures qui la relie, directement ou indirectement, à des clés déjà connues de l'utilisateur. Ce modèle évite la dépendance à une autorité centrale unique, mais peine à s'appliquer à grande échelle (le web grand public), car il exige un effort de vérification manuel et social difficile à industrialiser — ce qui explique pourquoi le modèle hiérarchique des PKI X.509, malgré ses propres limites, reste dominant pour sécuriser le web.

6.2 Certificate Transparency : détecter une émission frauduleuse

Certificate Transparency repose sur des journaux publics, structurés en arbres de Merkle (chapitre 3), dans lesquels chaque certificat émis par une AC participante doit être enregistré de façon vérifiable et infalsifiable a posteriori. Les navigateurs modernes exigent, pour certains types de certificats, une preuve d'inclusion dans au moins un journal reconnu avant d'accorder leur confiance. Ce mécanisme permet à un titulaire de domaine, ou à des chercheurs en sécurité, de surveiller ces journaux publics et de détecter rapidement l'émission d'un certificat non sollicité pour un domaine donné — une défense complémentaire face à une AC compromise ou négligente qui émettrait un certificat frauduleux sans que le titulaire légitime en soit informé par ailleurs.

7. Pièges d'implémentation courants

- **Vérifier la signature d'un certificat sans vérifier la chaîne complète jusqu'à une ancre de confiance**, ou accepter une AC intermédiaire non correctement liée à une racine reconnue.

- **Ignorer le résultat de la vérification de révocation** (CRL/OCSP), ou échouer silencieusement en cas d'indisponibilité du service de révocation plutôt que d'adopter une politique explicite (accepter ou refuser par défaut selon le niveau de risque).
- **Réutiliser un nonce ECDSA**, ou utiliser un générateur aléatoire de mauvaise qualité pour le produire — l'un des pièges les plus documentés en cryptographie appliquée, illustré par plusieurs compromissions réelles (section 3.1).
- **Confondre validation du certificat et validation du contenu applicatif** : un certificat valide prouve l'identité du serveur, pas la légitimité ou l'innocuité du contenu qu'il sert.

À retenir

- La signature numérique apporte la non-répudiation, absente d'un simple MAC (chapitre 3), grâce à l'asymétrie entre clé privée (signature) et clé publique (vérification).
- RSA-PSS est recommandé pour la signature RSA face à l'ancien schéma déterministe PKCS#1 v1.5.
- ECDSA exige un nonce aléatoire de haute qualité à chaque signature, sous peine d'exposer directement la clé privée ; Ed25519 (et RFC 6979 pour ECDSA) éliminent ce risque par construction déterministe.
- Une PKI repose sur une chaîne de confiance hiérarchique (racine hors ligne, intermédiaires en ligne) ; sa robustesse dépend entièrement de la rigueur de vérification à chaque maillon (identité, validité, révocation).
- OCSP stapling et Certificate Transparency renforcent la PKI classique en accélérant la détection de révocation et d'émission frauduleuse.
- Le Web of Trust offre une alternative décentralisée à la PKI hiérarchique, mais peine à s'appliquer à l'échelle du web grand public.

Chapitre 6 — Protocoles appliqués : TLS et cryptographie post-quantique

1. TLS (Transport Layer Security) : rôle et positionnement

TLS sécurise les communications entre un client et un serveur (HTTPS, mais aussi d'autres protocoles applicatifs) en fournissant confidentialité, intégrité et authentification du serveur (et optionnellement du client). Il illustre concrètement l'intégration de toutes les primitives vues dans ce module.

1.1 Position de TLS dans la pile réseau

TLS s'insère entre la couche transport (TCP) et la couche applicative : une application (navigateur web, client de messagerie, etc.) ouvre une connexion TCP classique, puis établit par-dessus une session TLS, avant d'échanger les données applicatives (requêtes HTTP, par exemple) qui transitent alors chiffrées et authentifiées de bout en bout de la connexion. Ce positionnement permet à TLS de protéger, de façon relativement transparente, n'importe quel protocole applicatif construit au-dessus (HTTPS pour HTTP, SMTPS pour le courrier électronique, etc.), sans que ce protocole applicatif ait lui-même à implémenter de mécanisme cryptographique.

2. La poignée de main TLS 1.3 (simplifiée)

1. **ClientHello** : le client propose les suites cryptographiques supportées et envoie des paramètres pour un échange de clé (ex. clé publique éphémère pour Diffie-Hellman sur courbe elliptique).
2. **ServerHello** : le serveur choisit une suite cryptographique, complète l'échange de clé et envoie son certificat.
3. **Vérification du certificat** : le client valide la chaîne de confiance jusqu'à une AC racine reconnue (chapitre 5).
4. **Dérivation des clés de session** : à partir du secret partagé issu de l'échange de clé, une fonction de dérivation de clé produit les clés symétriques utilisées pour le reste de la session.
5. **Communication chiffrée** : l'ensemble du trafic applicatif est protégé par un chiffrement symétrique authentifié (AES-GCM ou ChaCha20-Poly1305).

TLS 1.3 (2018) a simplifié la poignée de main par rapport à TLS 1.2, supprimé les suites cryptographiques jugées faibles (RC4, CBC sans protection adéquate, RSA pour l'échange de clé sans confidentialité persistante), et impose désormais un échange de clé Diffie-Hellman éphémère par défaut.

2.1 Ce que fait précisément l'étape de dérivation de clés

L'étape 4 illustre directement les chapitres précédents : le secret partagé brut issu de l'échange ECDHE (chapitre 4) n'est jamais utilisé tel quel comme clé de chiffrement. Il sert d'entrée à **HKDF** (chapitre 3, fondée sur HMAC), qui dérive à partir de ce secret plusieurs clés distinctes et

indépendantes : une clé pour chiffrer les données du client vers le serveur, une autre pour le sens serveur vers client, ainsi que des clés séparées pour certaines phases intermédiaires de la poignée de main elle-même (« clés de handshake », distinctes des « clés d'application » utilisées pour les données finales). Cette séparation stricte des clés par usage limite l'impact d'une éventuelle faiblesse localisée à un seul de ces flux.

2.2 Réduction du nombre d'allers-retours (1-RTT et 0-RTT)

Une des motivations principales de la refonte de TLS 1.3 était de réduire la latence d'établissement de connexion. TLS 1.2 nécessitait généralement deux allers-retours complets avant de pouvoir échanger des données applicatives chiffrées ; TLS 1.3 y parvient en un seul aller-retour (« 1-RTT »), le client envoyant directement ses paramètres d'échange de clé dans le `ClientHello` sans attendre de savoir quelle suite le serveur choisira. TLS 1.3 introduit également un mode optionnel **0-RTT**, permettant au client, lors d'une reconnexion à un serveur déjà visité, d'envoyer des données applicatives dès le premier message, en réutilisant un secret établi lors d'une session précédente. Ce gain de latence a un coût en sécurité qu'il faut connaître : les données envoyées en 0-RTT ne bénéficient pas de la même protection contre les attaques par rejeu (*replay*) qu'un flux 1-RTT classique, ce qui limite son usage recommandé à des requêtes idempotentes (sans effet de bord dupliqué en cas de rejeu).

3. Confidentialité persistante (Perfect Forward Secrecy)

Propriété garantissant que la compromission ultérieure de la clé privée à long terme d'un serveur ne permet pas de déchiffrer rétroactivement des sessions passées interceptées et enregistrées. Obtenue en utilisant des clés d'échange **éphémères** (générées pour chaque session puis détruites), typiquement via Diffie-Hellman éphémère sur courbe elliptique (ECDHE). C'est une des raisons pour lesquelles TLS 1.3 impose ce mécanisme par défaut.

3.1 Scénario illustratif

Imaginons qu'un adversaire enregistre passivement l'intégralité du trafic TLS chiffré échangé entre un client et un serveur sur plusieurs mois (scénario purement illustratif). Si le serveur utilisait un transport de clé RSA statique (ancien schéma TLS, aujourd'hui supprimé de TLS 1.3), la compromission ultérieure de la clé privée RSA du serveur — par exemple lors d'un incident de sécurité découvert bien plus tard — permettrait à l'adversaire de déchiffrer rétroactivement l'ensemble du trafic enregistré. Avec ECDHE, chaque session utilise une paire de clés éphémères générée puis détruite immédiatement après l'établissement du secret de session : la compromission ultérieure de la clé privée long terme du serveur (utilisée uniquement pour authentifier, c'est-à-dire signer, l'échange, et non pour le chiffrer directement) ne permet pas de reconstituer les secrets de session déjà détruits. C'est cette différence structurelle qui a motivé le retrait complet du transport de clé RSA dans TLS 1.3.

4. Suites cryptographiques et obsolescence

Une « suite cryptographique » combine un mécanisme d'échange de clé, un algorithme de signature, un chiffrement symétrique et une fonction de hachage. Les recommandations évoluent avec le temps à mesure que des faiblesses sont découvertes (rappel du module *Cryptanalyse*) : SSL 2.0/3.0, TLS

1.0/1.1, RC4, l'export de clés faibles (« EXPORT »), sont aujourd'hui déconseillés voire interdits par les référentiels de sécurité (ANSSI, PCI-DSS).

4.1 Composition d'une suite en TLS 1.3

TLS 1.3 a simplifié la notation des suites cryptographiques par rapport à TLS 1.2 : l'échange de clé et l'algorithme de signature sont désormais négociés séparément de la suite de chiffrement symétrique proprement dite, qui se limite à un couple algorithme AEAD + fonction de hachage (utilisée pour HKDF), par exemple `TLS_AES_256_GCM_SHA384` ou `TLS_CHACHA20_POLY1305_SHA256`. Cette simplification réduit fortement le nombre de combinaisons possibles par rapport à TLS 1.2, où le nombre très élevé de suites disponibles (certaines faibles) rendait la configuration serveur plus sujette à erreur.

```
# Lister les suites cryptographiques TLS 1.3 supportées par OpenSSL
openssl ciphers -s -tls1_3

# Se connecter à un serveur en forçant TLS 1.3 et afficher la suite négociée
openssl s_client -connect exemple.org:443 -tls1_3 -servername exemple.org </dev/null 2>
```

4.2 Bonnes pratiques actuelles de configuration serveur

- **Désactiver explicitement TLS 1.0/1.1 et SSL 2.0/3.0** côté serveur, même si le client ne les propose théoriquement jamais, pour éliminer tout risque de négociation dégradée forcée par un attaquant actif.
- **N'activer que des suites AEAD** (AES-GCM, ChaCha20-Poly1305), en écartant toute suite basée sur CBC sans protection adéquate ou sur RC4.
- **Activer HSTS** (*HTTP Strict Transport Security*) côté serveur web pour empêcher qu'un client accepte une connexion HTTP non chiffrée vers le même domaine après une première visite en HTTPS, réduisant l'exposition à une attaque de rétrogradation (*downgrade*).
- **Surveiller et renouveler les certificats avant expiration**, en automatisant ce renouvellement plutôt qu'en s'appuyant sur une procédure manuelle sujette à l'oubli.

5. Erreurs d'implémentation TLS fréquentes (audit)

- Ne pas vérifier le nom de domaine du certificat.
- Accepter des certificats auto-signés ou expirés en production.
- Désactiver la vérification de certificat pendant le développement, oubli en production (vulnérabilité réelle et récurrente).
- Utiliser des suites cryptographiques obsolètes non désactivées côté serveur.

Ces points recoupent directement la grille d'audit applicatif vue dans le module *Audit organisation et technique*.

5.1 Heartbleed : un exemple documenté d'impact d'une faille d'implémentation

La vulnérabilité **Heartbleed** (CVE-2014-0160), découverte en 2014 dans l'implémentation OpenSSL de l'extension *Heartbeat* de TLS, illustre bien la différence entre une faille de conception cryptographique et une faille d'implémentation logicielle : le protocole TLS lui-même n'était pas en cause, mais un défaut de vérification de longueur dans le code de traitement des messages Heartbeat permettait à un attaquant distant de lire, message après message, des fragments de la mémoire du serveur au-delà des données censées être renvoyées — pouvant potentiellement exposer des clés privées, des identifiants de session ou d'autres données sensibles présentes en mémoire au moment de l'attaque. Ce cas, largement documenté publiquement, rappelle qu'un protocole cryptographique correctement conçu reste entièrement dépendant de la rigueur de son implémentation logicielle : la meilleure spécification ne protège pas contre un bug d'implémentation dans le code qui la met en œuvre.

6. La menace post-quantique et la migration en cours

L'algorithme de Shor (rappel : module *Cryptanalyse*, chapitre 5), exécuté sur un ordinateur quantique suffisamment puissant, casserait RSA, Diffie-Hellman et ECC classiques. Bien qu'un tel ordinateur ne soit pas disponible aujourd'hui à l'échelle nécessaire, deux considérations motivent une action dès maintenant :

- « **Harvest now, decrypt later** » : un adversaire peut enregistrer aujourd'hui du trafic chiffré intercepté, dans l'espoir de le déchiffrer plus tard une fois l'informatique quantique suffisamment mature — un risque réel pour des données à confidentialité longue durée.
- **Délai de migration** : le déploiement d'une nouvelle famille cryptographique à l'échelle d'Internet prend typiquement plusieurs années.

Le NIST a standardisé en 2024 des algorithmes post-quantiques (**ML-KEM**, anciennement Kyber, pour l'échange de clé ; **ML-DSA**, anciennement Dilithium, pour la signature), dont l'intégration progressive dans TLS et d'autres protocoles est en cours, souvent sous forme **hybride** (combinant un mécanisme classique et un mécanisme post-quantique, pour ne pas dépendre uniquement de la maturité encore récente de ces nouveaux algorithmes).

6.1 Pourquoi le chiffrement symétrique est moins directement menacé

L'algorithme de Shor s'attaque spécifiquement aux problèmes structurés sur lesquels reposent RSA (factorisation) et Diffie-Hellman/ECC (logarithme discret) : ce sont des problèmes possédant une structure algébrique que l'algorithme quantique sait exploiter. Le chiffrement symétrique (AES) et les fonctions de hachage (SHA-2, SHA-3) ne reposent pas sur ce type de structure ; ils restent exposés uniquement à l'algorithme de Grover, qui offre au mieux une accélération quadratique de la recherche exhaustive (réduisant en théorie la sécurité effective d'AES-128 à un niveau comparable à AES-64, ce qui reste hors de portée pratique). C'est pourquoi les recommandations actuelles de transition post-quantique se concentrent sur l'échange de clé et la signature (RSA, Diffie-Hellman, ECC, ECDSA), tandis qu'il est généralement conseillé, par précaution, d'utiliser des tailles de clé symétrique déjà robustes (AES-256 plutôt qu'AES-128) pour les données dont la confidentialité doit être garantie sur le très long terme.

6.2 Familles d'algorithmes post-quantiques standardisés

Les algorithmes post-quantiques standardisés par le NIST reposent sur des familles mathématiques différentes de celles utilisées par RSA/ECC, réputées résistantes aux algorithmes quantiques connus à ce jour :

Algorithme (nom NIST)	Ancien nom	Usage	Famille mathématique
ML-KEM	Kyber	Encapsulation de clé (échange)	Réseaux euclidiens structurés (<i>lattices</i>)
ML-DSA	Dilithium	Signature numérique	Réseaux euclidiens structurés (<i>lattices</i>)
SLH-DSA	SPHINCS+	Signature numérique	Fonctions de hachage uniquement

La diversité de ces familles (réseaux euclidiens pour deux d'entre elles, hachage seul pour la troisième) répond à une logique de diversification du risque similaire à celle motivant la coexistence de SHA-2 et SHA-3 (chapitre 3) : si une faiblesse inattendue venait à être découverte dans une famille mathématique, une alternative reposant sur des fondements différents resterait disponible.

6.3 Mode hybride : pourquoi ne pas basculer directement au post-quantique

Un déploiement post-quantique **hybride** combine, pour un même échange de clé, un mécanisme classique (par exemple X25519) et un mécanisme post-quantique (par exemple ML-KEM), le secret final étant dérivé de la combinaison des deux. Tant qu'au moins un des deux mécanismes reste sûr, le secret combiné reste sûr. Cette approche prudente permet de commencer à déployer une protection contre la menace « harvest now, decrypt later » dès aujourd'hui, sans dépendre uniquement de la confiance encore relativement récente accordée à ces nouveaux algorithmes standardisés, dont l'examen public continue de s'approfondir avec le temps — exactement le même raisonnement de prudence collective qui, historiquement, a présidé à l'adoption progressive d'AES ou de SHA-3 après leurs propres processus de sélection publique.

7. Pièges d'implémentation courants

- **Laisser des suites cryptographiques obsolètes activées côté serveur** « au cas où », par excès de compatibilité avec d'anciens clients, plutôt que de les désactiver explicitement.
- **Désactiver temporairement la vérification de certificat pour faciliter le développement ou le débogage**, et oublier de réactiver cette vérification avant la mise en production — une source récurrente d'incidents réels.
- **Ne pas surveiller la date d'expiration des certificats**, provoquant des interruptions de service évitables lorsqu'un certificat expire sans avoir été renouvelé à temps.
- **Traiter la migration post-quantique comme un projet à reporter indéfiniment**, alors que le risque « harvest now, decrypt later » s'applique dès aujourd'hui aux données interceptées, même si leur déchiffrement effectif n'interviendrait que plus tard.

À retenir

- TLS combine, de façon opérationnelle, échange de clé asymétrique, authentification par certificat et chiffrement symétrique authentifié — la synthèse pratique de tout ce module.
- TLS 1.3 réduit la latence d'établissement de connexion (1-RTT, voire 0-RTT avec des précautions spécifiques contre le rejeu) tout en supprimant les suites cryptographiques jugées faibles.
- La confidentialité persistante (clés éphémères ECDHE) protège les échanges passés même en cas de compromission future d'une clé long terme, contrairement à l'ancien transport de clé RSA statique.
- Une faille d'implémentation (Heartbleed) peut compromettre la sécurité d'un protocole pourtant correctement conçu : la rigueur d'implémentation compte autant que la conception cryptographique elle-même.
- La transition post-quantique est engagée par anticipation, notamment face au risque « harvest now, decrypt later » sur des données sensibles à long terme ; elle cible en priorité l'échange de clé et la signature, moins directement menacés côté chiffrement symétrique.
- Le déploiement hybride (classique + post-quantique) permet une transition prudente, sans dépendre uniquement de la maturité encore récente des nouveaux algorithmes standardisés.